

immobiliare-photo-lab — nota di accompagnamento

Case study "Product Builder, Agentic AI Products" — Pio Santosuosso, settembre 2026 · prototipo: <https://immobiliare-photo-lab.vercel.app> · codice: <https://github.com/Pio57/immobiliare-photo-lab>

1. Il problema che affronto

Il caso d'uso è l'agente immobiliare che, in giornate piene di visite, fotografa in fretta un nuovo immobile con il cellulare, con la luce che c'è. Quelle foto finiscono direttamente nell'annuncio: quasi mai un fotografo passerà a rifarle.

Il bisogno è pubblicare subito senza che le foto sembrino fatte in fretta, senza pubblicarle così come sono né tornare a rifarle. L'obiettivo del prodotto è migliorare automaticamente le foto nei difetti tipici dello scatto veloce, lasciando all'agente il controllo e senza toccare la realtà dell'immobile.

Il prodotto riceve le foto, riconosce i difetti (foto buia, in controluce, colori falsati, rumore, inclinazione, compressione, bassa risoluzione) e corregge soltanto quelli, un intervento per difetto: una foto solo storta viene solo raddrizzata. Poi mostra all'agente cosa ha fatto; se non serve nulla, lascia l'originale.

Il prodotto non aggiunge, non toglie e non ridisegna nulla: niente cielo nuovo, oggetti rimossi, stanze allargate, crepe o muffa cancellate. Mosso, prospettiva e disordine li segnala, non li corregge. Le foto irrecuperabili le consiglia di rifare.

Il rischio principale da evitare è ingannare il compratore: la funzione non ha modo di far sparire o sistemare un dettaglio della casa, perché agisce solo su luce, colore, nitidezza e inclinazione, e ogni risultato viene confrontato con l'originale e scartato se se ne allontana troppo. Gli altri rischi (una correzione che peggiora la foto, un ritaglio non dichiarato, un agente che non sa cosa è cambiato) li affronto mostrando e spiegando ogni modifica prima della pubblicazione.

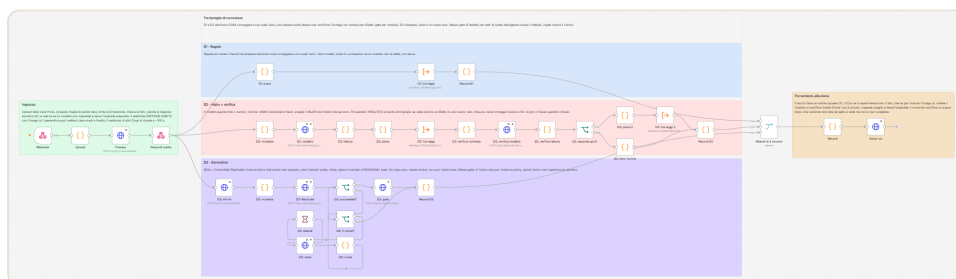
2. Come strutturo il workflow e i flussi da testare

Ho messo a confronto tre flussi, cioè tre modi diversi di decidere cosa correggere in una foto:

- **D1 — Regole.** Lavora solo sui numeri: luminosità media, dominante di colore, rumore e linee inclinate vengono misurati e un insieme di regole scritte a mano li traduce negli interventi da applicare. Nessun modello di intelligenza artificiale.
- **D2 — Modello esperto con verifica.** Un modello di visione (Claude Haiku) riceve la foto e le stesse misure, indica i difetti e consiglia interventi e valori; dopo la correzione riguarda il risultato accanto all'originale e, se non lo convince, corregge il proprio piano.
- **D3 — Modello generativo.** Un modello generativo (SDXL con ControlNet) riceve la foto e un prompt da fotografo esperto e restituisce direttamente una nuova immagine della stessa stanza, ben esposta, pulita e nitida.

I flussi vivono in quattro workflow n8n che si richiamano tra loro:

- **Correggi**, il motore delle correzioni: riceve una foto e un piano, applica un intervento alla volta (colore, luce, pulizia, raddrizzamento, nitidezza, risoluzione) e dopo ciascuno esegue il controllo di fedeltà, scartando l'intervento se il risultato si allontana troppo dall'originale.
- **Prodotto**, chiamato dal sito a ogni foto caricata: misura la foto, la manda in parallelo a D1, D2 e D3 (i primi due richiamano Correggi come sotto-workflow, il terzo chiama il modello generativo e applica lo stesso controllo) e salva un record con le tre versioni e tutto ciò che è successo.
- **Scelte**, che raccoglie le preferenze dei valutatori e restituisce i risultati aggregati al sito; **Batch**, che ripete il lavoro di Prodotto su un intero insieme di foto per preparare la sperimentazione.



Il workflow Prodotto in n8n: la foto entra, passa in parallelo dalle tre corsie D1, D2 e D3, e ne esce un record con le tre versioni.

Gli output sono quindi, per ogni foto, tre versioni con un registro di difetti trovati, interventi applicati o scartati, fedeltà, costo e tempo. I risultati li raccolgo con una sperimentazione cieca: un pool di agenti immobiliari come beta tester che, nella vista Studio, vedono l'originale e due versioni affiancate senza sapere quale flusso le ha prodotte e scelgono quella che userebbero come copertina, foto dopo foto. La stessa scelta avviene nella vista Prova sulla propria foto appena caricata. Ogni giudizio viene registrato e la vista Esperimento li aggrega in un tabellone. Con quali parametri leggere quei dati per scegliere il metodo migliore lo descrivo nel punto seguente.

3. Quali parametri uso per valutare che un metodo è migliore dell'altro

La sperimentazione non è ancora stata fatta: qui definisco cosa misurerei e come userei le misure per decidere. I parametri non pesano tutti allo stesso modo: uno è una soglia di ammissione, gli altri sono criteri di scelta in ordine di importanza.

- **Rischio di alterare l'immobile (fedeltà visiva).** È il primo parametro e non si ottimizza: si supera o si esce. Per un portale immobiliare la fedeltà è il fondamento della fiducia, e una foto che inventa o cancella un dettaglio distrugge la credibilità dell'annuncio. Misurerai, per ogni workflow, la quota di correzioni scartate dal controllo automatico di fedeltà e il tasso di alterazioni notate dai valutatori nel test cieco (punto 4). Un workflow con più del dieci per cento di correzioni scartate non è ammesso, qualunque sia la qualità delle sue foto.
- **Qualità dell'output.** È il valore del prodotto per l'agente. La misurerai con il test cieco: per ogni workflow, la quota di confronti vinti sul totale in cui è comparso, con un intervallo di confidenza che dice quanto fidarsi del numero con i giudizi raccolti.
- **Costo per immagine.** È ciò che incide sull'azienda, perché ogni annuncio ha tra le dieci e le venti foto e la piattaforma ne pubblica migliaia al giorno. Per un annuncio da venti foto, a prezzi di listino: D1 (regole) non fa chiamate e costa praticamente zero; D2 (modello esperto) fa due chiamate a Claude Haiku per foto, circa 0,004 dollari l'una, quindi 0,08 dollari per annuncio; D3 (generativo) con un servizio gestito di generazione immagini costa dalle dieci alle quindici volte tanto, circa 0,04–0,06 dollari a immagine (per esempio GPT Image a qualità media), quindi 0,80–1,20 dollari per annuncio; nel prototipo ho usato un modello open su GPU a consumo (SDXL su Replicate), che costa solo 0,003 dollari a immagine ma senza garanzie di servizio. Su diecimila annunci al giorno: nulla per D1, circa 800 dollari al giorno per D2, da 600 a 12.000 per D3 a seconda del fornitore.
- **Affidabilità del workflow.** Conta molto meno: la misurerai (risposte illeggibili del modello, ricadute sulla riserva, correzioni imposte dal sistema al piano del modello) e la userai solo per escludere un workflow che fallisce con frequenza.
- **Tempo di elaborazione.** Non lo considero: la correzione si fa una volta sola per annuncio e può girare in background, pubblicando le foto quando sono pronte; dieci o quaranta secondi a foto sono invisibili all'agente.

La decisione procede a imbuto, con la regola fissata prima di guardare i dati. Primo filtro, il rischio: restano solo i workflow sotto la soglia. Secondo, tra i rimasti conta la preferenza nel test cieco, con almeno trenta giudizi. Terzo, se due workflow non sono distinguibili sulla qualità, decide il costo, con una considerazione in più: le regole costano zero ma sono rigide, e ogni nuovo tipo di difetto richiede nuove regole, mentre il modello esperto si adatta e si mantiene con un prompt; se la differenza di costo è piccola rispetto al valore del prodotto, l'adattabilità può giustificarla. Il tabellone della vista Esperimento applica già questa regola ai giudizi raccolti.

4. Come misuro gli elementi qualitativi

Distinguo due fasi: una misurazione controllata sul prototipo, per scegliere il workflow prima di investire traffico reale, e una in produzione, dove i giudizi diventano azioni degli utenti.

Fase 1 — sul prototipo, test cieco. Un panel di dieci-quindici persone, idealmente con qualche agente immobiliare, usa il sito senza sapere quale workflow ha prodotto cosa: con ventiquattro foto e dieci tester, duecentoquaranta giudizi per misura. Ogni elemento qualitativo diventa una domanda precisa e un numero confrontabile tra D1, D2 e D3:

- *Realismo.* Originale accanto a una versione elaborata, una alla volta: "vedi elementi finti, generati o strutturalmente diversi rispetto all'originale?" (sì/no). Il conteggio è il tasso di alterazione; vince chi sta più vicino a zero. È la misura che tutela il vincolo del brief.
- *Qualità.* La sola versione elaborata, senza originale: voto da 1 a 5 su luce, nitidezza e colori. La media dei voti (mean opinion score) dà un punteggio per workflow; sopra 3,5 la foto è percepita come buona.
- *Foto migliore.* Originale in alto e le tre versioni affiancate, anonime: "se fossi l'agente, quale useresti?". Il conteggio è la quota di vittorie per workflow: l'equilibrio tra difetto corretto e naturalezza conservata.
- *Output efficace.* Giudico io con le etichette manuali dei difetti: per ogni foto, il workflow ha risolto i difetti presenti senza introdurne di nuovi? Il conteggio è la quota di foto pubblicabili senza altro ritocco.

Fase 2 — in produzione, test A/B. Se il prototipo indica un workflow, lo si prova sul mercato: annunci con foto elaborate contro annunci con foto originali, a parità di zona e fascia di prezzo, con giudizi impliciti. Il realismo si misura al contrario, con le segnalazioni "l'immobile non corrisponde alle foto" e il feedback degli agenti dopo le visite: un picco è un fallimento. Qualità e foto migliore si misurano con il tasso di click dai risultati all'annuncio quando la copertina è elaborata, con il tempo in galleria e con i contatti ricevuti. L'efficacia si misura sull'agente: tempo risparmiato tra scatto e pubblicazione e quante volte scarta la versione proposta per ricaricare l'originale.

Il prototipo copre già la prima fase con il test a coppie della vista Studio e con il controllo automatico di fedeltà; la domanda sul realismo e il voto di qualità sono l'estensione naturale dello stesso sito.

Prototipo, strumenti e limiti

Il sito (React, su Vercel) parla con quattro workflow n8n self-hosted su un server; correzioni, controllo di fedeltà e dati stanno in un servizio Python con OpenCV sullo stesso server. Modelli: Claude Haiku per diagnosi e verifica, SDXL con ControlNet e Real-ESRGAN su Replicate. Sviluppo con Claude Code. È tutto reale: chiamate ai modelli, le ventiquattro foto (case vere, da annunci pubblici), il controllo di fedeltà e la sua calibrazione, i giudizi, i costi e i tempi. È simulato il numero dei valutatori, finora uno: la sperimentazione con gli agenti è la proposta, non il risultato. Limiti principali: il controllo di fedeltà non vede strutture sotto i tre pixel né la super-risoluzione, che per questo viene etichettata; il rilevatore di inclinazione rinuncia quando le linee stanno da un lato solo; il campione di foto è piccolo e poco difettoso. Prossimi passi: la sperimentazione cieca con un gruppo di agenti; un modulo per sfocare volti e targhe; il test sulla piattaforma, foto corrette contro originali, misurato sui contatti.